



Pandas와 데이터 전처리

Pandas

❖Pandas의 필요성:

- NumPy 배열은 빠르고 강력하지만, 각 열이 무엇을 의미하는지 알 수 없다.
- NumPy 위에 구축되었으며, 레이블(인덱스, 컬럼명)이 있는 데이터를 다루는 데 특화.
- 엑셀 스프레드시트나 데이터베이스 테이블을 파이썬으로 가져온 것이라 생각하면 쉽다.

❖Pandas

- Python Data Analysis Library
- R 프로그래밍 언어의 data.frame을 본떠 만들었음.
- 행과 열로 이루어진 표(tabular) 형식의 데이터를 쉽고 직관적으로 다룰 수 있도록 도와주는 도구
- 핵심 데이터 구조: Series와 DataFrame

Pandas 주요 기능

- ❖데이터 읽기/쓰기 : CSV/Excel/JSON/SQL 등 다양한 포맷의 파일 처리
- ❖데이터 선택 및 필터링 : 특정 조건에 맞는 행이나 열을 추출
- ❖데이터 정렬 : 특정 열을 기준으로 오름차순/내림차순으로 정렬.
- ❖결측치 처리 : 비어있는 값을 찾고, 채우거나 제거
- ❖데이터 그룹화 및 집계 : 특정 그룹별로 묶어 평균, 합계, 개수 등을 계산
- ❖데이터 병합 및 연결 : 여러 개의 DataFrame을 하나로 합침

→ 데이터 처리의 거의 모든 과정을 지원

Series: 1차원 데이터

❖개념:

- 1차원 배열과 같은 구조로, 하나의 열(column)에 해당하는 데이터를 담음
- 각 데이터는 고유한 인덱스(index)를 가짐

❖생성:

```
import pandas as pd

# 리스트를 이용해 Series 생성
s = pd.Series([10, 20, 30, 40], name='Score')
print(s)
```

```
0    10
1    20
2    30
3    40
Name: Score, dtype: int64
```

❖주요 속성:

- series.values (NumPy 배열로 반환)
- series.index

DataFrame: 2차원 데이터

❖개념:

- 2차원 테이블 구조로, 여러 개의 Series가 모여 만들어진 형태
- 입니다. 행(row)과 열(column) 그리고 인덱스(index)로 구성됨
- 엑셀 시트를 생각하면 이해하기 쉬움

❖생성:

```
import pandas as pd

# 딕셔너리를 이용해 DataFrame 생성
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [25, 30, 35, 40],
        'City': ['Seoul', 'Busan', 'Incheon', 'Daegu']}
df = pd.DataFrame(data)
print(df)
```

	Name	Age	City
0	Alice	25	Seoul
1	Bob	30	Busan
2	Charlie	35	Incheon
3	David	40	Daegu

❖주요 속성:

- df.columns, df.index, df.values, df.shape, df.dtypes

실습 1: Series와 DataFrame 직접 만들어보기

- ❖ Series와 DataFrame 직접 만들어보기.
- ❖ Series와 DataFrame 속성 확인하기.

데이터 불러오기: pd.read_csv()

- 데이터 분석의 시작점과도 같은, 가장 기본적이고 중요한 함수
- CSV 파일을 읽어서 Pandas DataFrame으로 변환하는 역할
- 기본 사용법:
 - 파일의 첫 번째 줄을 자동으로 '열 이름(header)'으로 인식하고,
 - 행의 '이름(index)'은 0부터 시작하는 숫자로 자동 생성

```
import pandas as pd

# 'my_data.csv' 파일을 읽어와서 df라는 DataFrame으로 만듭니다.
df = pd.read_csv('my_data.csv')

# 불러온 데이터의 첫 5행을 확인합니다.
print(df.head())
```

데이터 불러오기: `pd.read_csv()`

자주 쓰이는 옵션(파라미터)

- `filepath_or_buffer` : 파일 경로
- `sep` (or `delimiter`)
 - 구분자. 데이터가 어떤 기호로 분리되어 있는지 알려줌. CSV는 'Comma(,) Separated Values'의 약자지만, 가끔 탭(`\t`), 세미콜론(`;`), 공백 등으로 분리된 파일도 있다
 - 예시: `pd.read_csv('data.tsv', sep='\t')`
- `encoding`
 - 한글 깨짐 방지용, `cp949` 또는 `utf-8`
 - 예시: `pd.read_csv('data_korean.csv', encoding='cp949')`

데이터 불러오기: `pd.read_csv()`

자주 쓰이는 옵션(파라미터)

- `index_col`
 - 파일에 있는 특정 열을 DataFrame의 행 이름(index)으로 바로 지정하고 싶을 때 사용합니다.
 - 예시: `pd.read_csv('data.csv', index_col='StudentID')` #또는 `index_col=0` (0번째 열을 인덱스로)
- `header`
 - 열 이름(header)으로 사용할 행의 위치를 지정합니다. 만약 파일에 열 이름이 없다면 `header=None`으로 설정
 - 예시: `pd.read_csv('no_header_data.csv', header=None)`
- `names`
 - 설명: `header=None`일 때, 열 이름을 직접 리스트 형태로 만들어줄 수 있습니다.
 - 예시: `pd.read_csv('no_header_data.csv', header=None, names=['이름', '나이', '도시'])`

데이터 훑어보기 (Getting a First Look)

❖ df.head(n):

- 데이터의 첫 n행을 보여준다.
- "데이터가 어떻게 생겼는지" 감을 잡는 용도.

❖ df.tail(n):

- 마지막 n행을 보여준다.
- 데이터가 순서대로 정렬되어 있는지 등을 확인할 때 유용.

❖ df.sample(n):

- 무작위 n행을 보여준다.
- 데이터가 편향되지 않았는지 확인하는 데 도움.

데이터의 구조와 요약 정보 확인

❖ df.info() :

- 전체 행의 개수와 인덱스 범위.
- 각 열의 Non-Null 데이터 개수
- 데이터 타입(Dtype)
- 메모리 사용량

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
8   class       891 non-null    object
9   who         891 non-null    object
10  adult_male   891 non-null    bool
11  deck         203 non-null    object
12  embark_town  889 non-null    object
13  alive        891 non-null    object
14  alone        891 non-null    bool
dtypes: bool(2), float64(2), int64(4), object(7)
memory usage: 92.4+ KB
```

데이터의 구조와 요약 정보 확인

❖ df.describe():

- 데이터에 대한 핵심 통계량을 한 번에 보여줌

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

데이터의 구조와 요약 정보 확인

- 숫자 데이터를 가진 열에 대해 다음 8가지 정보를 계산
 - `count` : 결측치(NaN)를 제외한 데이터의 개수입니다.
 - `mean` : 데이터의 평균값입니다.
 - `std` : 표준편차(Standard Deviation)입니다. 데이터가 평균으로부터 얼마나 퍼져 있는지를 나타내며, 값이 클수록 데이터가 넓게 분포하고 있다는 의미입니다.
 - `min` : 가장 작은 값(최솟값)입니다.
 - `25%` (1사분위수): 전체 데이터를 크기순으로 정렬했을 때, 하위 25%에 해당하는 값입니다.
 - `50%` (2사분위수/중앙값): 전체 데이터의 중앙에 위치하는 값(Median)입니다. 평균(mean)보다 이상치(outlier)의 영향을 덜 받는 특징이 있습니다.
 - `75%` (3사분위수): 전체 데이터의 상위 25% (하위 75%)에 해당하는 값입니다.
 - `max` : 가장 큰 값(최댓값)입니다.
- 💡 `25%` , `50%` , `75%` 값들을 통해 데이터가 어떤 범위에 집중되어 있는지 분포를 짐작할 수 있습니다.

데이터의 구조와 요약 정보 확인

- 문자열 데이터를 가진 열에 대해 다음 4가지 정보를 계산
 - `describe()` 함수에 `include='object'` 옵션을 추가해야 문자열(object 타입) 데이터에 대한 요약 정보를 볼 수 있다.

```
df.describe(include='object')
```

- `count` : 결측치를 제외한 데이터의 개수입니다.
- `unique` : 고유한 값(카테고리)의 개수입니다.
- `top` : 가장 자주 등장하는 값입니다.
- `freq` : `top` 값이 등장한 빈도수(횟수)입니다.

실습 2: 데이터 로딩 및 기본 탐색 (EDA Part 1)

❖ CSV 파일에서 데이터 불러오기

- 실습 데이터: 교육용으로 아주 유명한 타이타닉(Titanic) 데이터셋을 사용
- `pd.read_csv()` : 웹에 있는 CSV 파일을 DataFrame으로 바로 불러옴

❖ 데이터 훑어보기

- `df.head()`: 데이터의 첫 5개 행을 출력 → 전체적인 구조를 파악
- `df.tail()`: 마지막 5개 행 출력
- `df.sample(5)`: 무작위로 5개 행을 추출 → 데이터가 편향되어 있는지 확인

실습 2: 데이터 로딩 및 기본 탐색 (EDA Part 1)

❖데이터의 구조와 요약 정보 확인하기

- `df.info()`:
 - 데이터프레임의 요약 정보를 보여줌.
 - 각 열의 데이터 타입과 결측치(비어있는 값)가 몇 개인지 파악하는 데 가장 중요
- `df.describe()`:
 - 숫자형 데이터들의 기초 통계량(평균, 표준편차, 최소/최대값 등)을 보여줌.
 - 데이터의 분포를 짐작할 수 있다.

결측치 (Missing Value)

❖ 결측치란?

- 데이터셋에서 값이 누락된 상태
- 설문조사에서 응답자가 특정 질문에 답하지 않거나, 시스템 오류로 데이터가 누락되는 등 다양한 이유로 발생할 수 있음
- 결측치가 있는 데이터를 그대로 분석에 사용하면 왜곡된 결과를 초래할 수 있음

❖ 결측치 식별

- NumPy 데이터 유형의 경우, `np.nan`이 누락된 데이터
- `df.isnull()` / `df.isna()` : 결측치 위치에 `True`를 반환하는 DataFrame을 생성
- `df.isnull().sum()` : 열(Column)별 결측치 개수 # 가장 일반적인 방법

결측치 처리 전략 1: 제거 (dropna)

❖ `df.dropna()`:

- 결측치가 하나라도 있는 행을 모두 제거.
- `axis=1` 옵션: 결측치가 있는 열을 제거.
- 언제 사용하는가?
 - 데이터가 충분히 많고,
 - 결측치가 있는 행의 비율이 낮을 때.
 - 또는 특정 열이 거의 다 비어있을 때.

결측치 처리 전략 2: 채우기 (fillna)

❖ `df.fillna(0):`

- 특정 값으로 채우기.
- `df['Age'].fillna(df['Age'].mean()):`
 - 다른 값(평균, 중앙값, 최빈값 등)으로 채우기. → 가장 일반적인 결측치 처리 방법
- 어떤 값으로 채울까?
 - 숫자형 데이터: 평균(mean) 또는 중앙값(median).
 - 범주형 데이터: 최빈값(mode).

불필요한 데이터 제거 (drop)

❖ `df.drop('column_name', axis=1):`

- 특정 열(들)을 제거.

❖ `df.drop(index_number):`

- 특정 행을 제거.

❖ 왜 제거하는가?

- 분석에 필요 없는 열(e.g., ID),
- 중복된 정보,
- 너무 많은 고유값을 가져 모델 성능에 방해가 되는 열 등을 제거.

inplace 옵션

❖ inplace=False (기본값)

- 원본은 그대로 두고, 변경된 새로운 DataFrame을 반환
- 변경된 결과를 사용하려면 새로운 변수에 할당하거나, 원래 변수에 덮어써야 함

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'A': [1, 2, np.nan], 'B': [4, np.nan, 6]})

# 원본 df는 바뀌지 않음
new_df = df.fillna(0)

print("--- 원본 df ---")
print(df)
print("\n--- new_df (결과) ---")
print(new_df)
```

--- 원본 df ---

	A	B
0	1.0	4.0
1	2.0	NaN
2	NaN	6.0

--- new_df (결과) ---

	A	B
0	1.0	4.0
1	2.0	0.0
2	0.0	6.0

inplace 옵션

❖ inplace=True

- 원본 DataFrame을 직접 수정하고, 아무것도 반환하지 않음 (None)
- 변수에 다시 할당할 필요가 없음

```
df = pd.DataFrame({'A': [1, 2, np.nan], 'B': [4, np.nan, 6]})
```

```
# df 자체가 수정됨, 반환값은 None  
result = df.fillna(0, inplace=True)
```

```
print("--- 원본 df (수정됨) ---")  
print(df)  
print(f"\n--- 반환값 ---")  
print(result)
```

--- 원본 df (수정됨) ---

	A	B
0	1.0	4.0
1	2.0	0.0
2	0.0	6.0

--- 반환값 ---
None

과거에는 inplace=True가 메모리를 절약한다는 장점 때문에 종종 사용되었지만, 최근에는 inplace=False (기본값) 사용이 더 권장

실습 3: 결측치 처리하기

❖ 열(Column)별 결측치 개수 확인하기

- `isnull()`과 `sum()`을 연달아 사용하면 각 열에 결측치가 총 몇 개 있는지 확인 가능
- 'age', 'deck', 'embarked' 열의 결측치 개수 확인하기

❖ 'age':

- 일부만 비어있으므로, 임의의 값을 채워주는 게 좋다.
- 가장 흔한 방법은 전체 승객의 평균 나이로 채우는 것.
- 평균 또는 중앙값으로 `fillna()`를 사용해 채워보기.

실습 3: 결측치 처리하기

❖ 'deck':

- 결측치가 너무 많아서 유의미한 정보를 얻기 어려움
- 이럴 땐 과감히 열 자체를 삭제하는 것이 좋은 전략
- `drop()` 함수를 사용하고, 열을 삭제한다는 의미로 `axis=1` 옵션을 줌

❖ 'embarked'와 'embark_town'

- 이 두 열은 결측치가 매우 적음.
- 이런 범주형 데이터는 가장 많이 나타나는 값, 즉 최빈값으로 채우는 것이 일반적
- `mode()[0]`를 사용해 최빈값을 찾아 `fillna()`를 사용해 채워보기

❖ 다시 `info()`를 실행하여 모든 열의 Non-Null 개수가 같아졌는지 확인